

ReconDreamer-RL Framework 技术架构详解

日期：2026-04-03

范围：/root/clone/ReconDreamer-RL/framework 以及主入口 script/train_actor_learner_v2.py

文档类型：内部技术架构说明

适用对象：框架研发、算法研发、训练系统维护

执行摘要

ReconDreamer-RL/framework 当前已经具备一个完整 actor-learner

强化学习训练底座的核心形态。它的价值不在于单个 PPO 或 ReinforcePP

公式本身，而在于已经把自动驾驶闭环训练中最容易分散的工程职责收拢成了统一主链路：角色拉起 ->

环境采样 -> shard 落盘 -> batch 构建 -> Lightning update -> checkpoint/version 发布 -> actor 热更新。

从工程实现上看，这套框架已经明确了以下几件事：

- runner/ 负责系统启动、角色调度和运行时装配。
- agent/ 负责屏蔽不同策略模型的内部差异。
- io/ 负责 actor 和 learner 之间的文件协议与版本协同。
- batch/ 与 algorithms/ 负责从轨迹数据到训练目标的数学转换。
- lightning/ 负责训练生命周期、优化器、日志和版本发布。

这意味着后续无论是补新模型、补新算法，还是增强训练稳定性，都可以在既有边界内增量推进，而不需要重新改写整个训练主链路。

阅读导航

如果只需要快速建立整体认知，建议优先阅读以下章节：

- 第 2 章：系统总览
- 第 5 到 7 章：Actor、Buffer、Learner 主链路
- 第 8 到 10 章：batch、training_step 与版本发布
- 第 13 到 14 章：协议约束、局限和后续演进建议

如需定位具体实现，请重点关注文中嵌入的代码片段，它们对应的是当前框架最关键的工程落点，而不是辅助性细节。

1. 文档目的

系统说明 ReconDreamer-RL 当前 framework/ 的技术架构、模块边界、数据流、训练协议、关键代码路径以及后续可扩展点。文档重点覆盖当前正在使用的文件缓冲 actor-learner 主链路，而不是历史实验脚本或孤立工具模块。

从工程实现上看，这套框架是一套围绕自动驾驶策略训练构建的异步 RL 训练底座。它解决的问题包括：

- 如何把具体策略模型适配成统一的 Agent。
- 如何把底层仿真环境包装成 RL 可以采样的环境。
- 如何把 Actor 采样结果稳定写入共享缓冲区。
- 如何让 Learner 选择、消费、训练并回收 shard。
- 如何把新权重发布给 Actor，并保证版本切换有明确协议。
- 如何在 PyTorch Lightning 生命周期内承接 RL 的 update 语义。

当前主入口为 train_actor_learner_v2.py。

2. 系统总览

2.1 训练目标

framework/ 的核心目标，是把自动驾驶策略模型接入统一 RL 训练闭环，并允许 PPO、ReinforcePP 等算法在闭环仿真环境中持续执行“采样-训练-发版-继续采样”的循环。

2.2 当前主链路

当前代码主路径可以概括为：

配置 -> runner 组装 -> orchestrator/actor/learner 角色拉起 -> actor 采样 -> shard 落盘 -> learner 选 shard -> batch 构建 -> Lightning 训练 -> checkpoint/version 发布 -> actor 热更新权重

如果映射到文件，大致如下：

1. train_actor_learner_v2.py 负责读 YAML 和路由运行角色。
2. config_normalization.py 规范化 actor-learner 配置。
3. orchestrator.py 负责拉起 learner 与多个 actor 子进程。
4. actor_runtime.py 驱动 Actor 主循环。
5. collector.py 负责把环境交互整理为 shard。
6. buffer.py 负责 shard、版本号和停止信号等文件协议。
7. actor_learner_datamodule.py 等待并选择 shard，构建训练 batch。
8. actor_learner.py 从 shard 计算 return、advantage、old_value 等训练张量。
9. trajectory_module.py 在 training_step 中执行 PPO 或 ReinforcePP 损失计算。
10. actor_learner_module.py 在 update 结束后移动 shard、保存权重并推进版本号。

2.3 架构特征

当前架构有四个很鲜明的工程特征：

- actor 和 learner 解耦，通过文件缓冲区通信。
- 版本切换基于 weights/latest.ckpt 与 weights/version.txt 两个共享协议文件。
- 算法目标函数与训练循环调度分离，前者在 algorithms/，后者在 lightning/。
- 模型后端通过 agent/ 统一抽象，避免模型内部细节污染主训练链路。

3. 顶层目录与职责边界

3.1 runner：训练系统总调度层

framework/runner/ 负责把整个系统真正运行起来。它不负责目标函数，也不直接实现 shard 协议细节，而是负责：

- 配置规范化。
- 角色划分与进程拉起。
- Agent、环境和算法规格对象的装配。
- Learner 的 Lightning handoff。
- Actor 的 GPU 分配与环境变量准备。

这一层最重要的价值是把“训练怎么启动”与“训练中具体怎么算”分开。

3.2 agent：模型适配层

framework/agent/ 把具体策略模型抽象为统一 Agent 接口。Agent 协议定义在 base.py，核心方法包括：

- act()：单环境采样动作。
- act_batch()：多环境并行采样。
- logp_from_replay() / logp_from_replay_batch()：Learner 侧基于 replay 重算当前策略下的 log-prob。
- save_checkpoint() / load_checkpoint()：权重存取。
- wrap_ddp()：多卡训练时的 DDP 包装入口。

这使得 DiffusionDriveV2、SparseDrive、SparseDriveV2 以及测试用 DummyPolicy 都能走同一条主训练链路。

3.3 env_wrapper：环境包装层

framework/env_wrapper/ 负责把底层重建仿真环境包装成 RL 环境，输出统一的 reset() 与 step() 语义，并把奖励、碰撞、终止判定等逻辑组合进来。Actor 不直接感知底层仿真复杂性，而是只与环境包装层交互。

3.4 rollout：采样层

framework/rollout/ 是 actor 侧轨迹采样入口，负责把一段 horizon 内的观测、奖励、done、replay 等信息收集并打包为单个 shard。它是环境侧与 learner 侧之间的第一座桥。

3.5 io：协议与文件缓冲层

framework/io/ 是 actor-learner 异步协作的关键基础设施，负责：

- buffer 目录结构。
- shard 文件命名与落盘。
- consumed 回收。
- STOP 信号。
- latest checkpoint 与 version 管理。
- learner 侧 shard 筛选策略。

这里的实现细节不复杂，但它定义了整个系统的训练契约。

3.6 batch：shard 到训练 batch 的转换层

framework/batch/ 把多个 shard 转换成 learner 可直接训练的张量 batch。这里完成：

- reward、done 到 return 的转换。
- PPO 场景下的 value 估计与 GAE。
- advantage 标准化。
- replay 列表与 obs 对齐。

3.7 algorithms：目标函数层

framework/algorithms/ 负责 PPO / ReinforcePP 目标函数与算法规格对象，不再负责训练循环。当前 canonical 的损失实现集中在 trajectory_policy_core.py。

3.8 lightning：训练生命周期层

framework/lightning/ 把 RL update 接入 PyTorch Lightning：

- trajectory_datamodule.py 管理 minibatch 读取。
- trajectory_module.py 管理单个 minibatch 的训练步。
- actor_learner_datamodule.py 管理每个 update 前的 shard 选择与 batch 准备。
- actor_learner_module.py 管理 update 完成后的消费、保存、发版与日志。

3.9 rewards：奖励计算层

framework/rewards/ 提供奖励项计算，目前主流程走 tracking.py，将位置误差、航向误差、碰撞、jerk 和终止惩罚统一转换成标量 reward。

4. 从入口到角色拉起

4.1 顶层入口脚本

train_actor_learner_v2.py 做了三件事：

1. 读取 YAML 配置。

2. 动态导入 actor_main、learner_main、orchestrator_main 与 normalize_actor_learner_cfg。

3. 根据 --role 将进程路由到 orchestrator、actor 或 learner。

这个入口脚本非常薄，框架已经把主要职责沉到底层模块，而不是把逻辑堆在启动脚本里。

下面是入口路由的核心代码摘录，省略了参数定义和导入细节：

```
def main() -> None:    torch.set_float32_matmul_precision("high")    args = parse_args()    cfg = load_yaml()
```

这段代码说明当前主入口的设计原则非常明确：入口脚本只负责读取配置和做角色分发，不负责真正的训练逻辑。系统的主复杂度被下沉到了 runner/、lightning/ 和 io/，而不是在入口脚本中逐渐膨胀。

4.2 配置规范化

config_normalization.py 负责对 actor-learner 配置进行补全。它最关键的两个动作是：

- 如果设置了 actor_gpu_pool 与 actors_per_gpu，自动展开为 actor_gpu_ids。
- 如果启用了 auto_max_inflight_per_actor，则根据 shards_per_update / num_actors 自动推导 max_inflight_per_actor，避免 learner 等 shard 时出现容量不足。

4.3 orchestrator 的职责

orchestrator.py 是训练主控进程。它会：

- 创建 BufferPaths 并初始化 buffer 目录。
- 清理残留的 STOP 和 TRAINING_LOCK。
- 调用 resolve_actor_gpu_ids() 计算 actor 的 GPU 分配计划。
- 预热 gsplat CUDA 扩展。
- 拉起一个 learner 子进程。
- 拉起多个 actor 子进程。
- 持续监控子进程退出状态。
- 在 finally 中写 STOP，并等待 actor、learner 退出。

这意味着 orchestrator 的主要职责是“进程编排和生命周期托管”，而不是训练本身。

5. Actor 侧执行路径

5.1 Actor 启动

actor_runtime.py 中的 actor_main() 是 Actor 的主循环。其初始化过程包括：

- 从 actor_learner 配置读取 mode、actor_horizon、max_inflight_per_actor、poll_interval_s 等关键参数。
- 根据 gpu_id 选择执行设备。
- 通过 build_agent() 构建策略 Agent。
- 如果 learner 已经发布过权重，则读取 version.txt 并从 latest.ckpt 加载最新权重。
- 构建 actor 对应的环境实例。

5.2 Actor 的运行控制逻辑

Actor 主循环里有四个非常重要的控制点：

1. STOP 检查

只要检测到 STOP 文件，Actor 就会主动退出。

2. TRAINING_LOCK 检查

如果 actor 和 learner 在同一块 GPU 上，并且配置了 `pause_actor_on_learner_gpu=true`，Actor 会在 learner 训练期间暂停，降低显存竞争。

3. backpressure

Actor 会通过 `count_inflight()` 统计自己尚未被消费的 shard 数。如果超过 `max_inflight_per_actor`，则轮询等待，防止 buffer 无限制膨胀。

4. 热更新权重

Actor 会持续对比 `version.txt` 和本地 `local_ver`。一旦 learner 发布了更高版本，就自动执行 `agent.load_checkpoint()` 完成热更新。

Actor 主循环里最关键的代码可以概括为下面这段：

```
while True:    if stop_requested(paths):        stage(f"[actor{actor_id}] stop requested; exiting")
```

这段逻辑把 Actor 的运行语义说得很清楚：Actor 不是“无脑一直采样”，而是在 STOP、TRAINING_LOCK、buffer 背压和版本热更新四个控制面之下运行。也正因为这四个控制点存在，当前系统才能多进程异步采样下保持可控。

5.3 单环境采样与 shard 生成

当 `num_envs_per_actor == 1` 时，Actor 调用 `collector.py` 里的 `collect_single_env_shard()`。该函数在每个 step 内执行：

- 使用 `agent.act(obs)` 采样动作、旧 logp 与 replay。
- 调用环境 `env.step(action)` 得到 reward、terminated、truncated。
- 保存当前 obs、old_logp、reward、done、replay。
- 如果 episode 结束则立即 `env.reset()`，但 shard 仍继续采样直到凑够 horizon。

最终单个 shard 包含的关键字段有：

- obs
- old_logp
- reward
- done
- terminated
- truncated
- next_obs
- done_last
- terminated_last
- replay

- meta
- 可选的 next_value_feature

其中 meta 包含：

- actor_id
- env_id
- horizon
- weights_version
- time
- shard_idx

这些字段共同定义了 learner 能看到的训练数据视图。

下面是 shard 组装的核心代码摘录：

```
shard = {      "obs": torch.stack(obs_buf, dim=0),      "old_logp": torch.stack(old_logp_buf, dim=0).view(-1),
```

这段代码很重要，因为它几乎就是当前 actor-learner 协议的数据面定义。文档里后续提到的 GAE、old/new log-prob 对比、version 过滤、replay 重放，全部建立在这组字段之上。

5.4 shard 文件命名

Actor 把 shard 原子性地写到 buffer/shards/ 下，文件名模式类似：

```
actor{actor_id}_e{env_id}_v{local_ver}_t{timestamp}_{uuid}.pt
```

这里包含了 actor 身份、环境编号、权重版本和采样时间。Learner 的版本筛选逻辑依赖其中的 _v{version} 片段。

5.5 sync 与 async 模式

Actor 支持两种同步方式：

- async：写完 shard 后立刻继续下一轮采样。
- sync：写完 shard 后调用 wait_for_version()，直到 learner 把版本推进到 local_ver + 1。

因此，sync 本质上是“每个 update 后全体 actor 等待 learner 发版”，而 async 是“actor 持续采样，learner 自主择机消费”。

6. 文件缓冲与协议层

6.1 BufferPaths 约定

buffer.py 中的 BufferPaths 定义了整个 actor-learner 文件协议：

- buffer/shards/：待消费 shard。
- buffer/consumed/：已消费 shard。
- weights/latest.ckpt：最新权重。

- `weights/version.txt`：最新权重版本。
- `STOP`：停止训练广播信号。

这是当前主训练路径的协议基石。

文件协议的核心定义如下：

```
@dataclassclass BufferPaths:    root: str    @property    def shards_dir(self) -> str:        return os.path
```

这段实现表明，当前系统采用的是非常朴素但非常清楚的文件协议。好处是目录一眼就能看懂，排障时不需要先理解复杂服务端状态机；代价是吞吐、调度灵活性和元数据管理能力都受文件系统能力限制。

6.2 原子写入

`atomic_torch_save()` 与 `write_int()` 都采用“先写临时文件再 `os.replace`”的方式完成原子替换。这样可以避免 learner 读取到半写入状态的 shard 或版本文件。

6.3 inflight 与 BackPressure

`count_inflight()` 按 `actor_id` 统计当前 `buffer/shards/` 中仍未被 learner 消费的 shard 数。Actor 用它做 `backpressure`，Learner 不直接管每个 actor 的节奏，但通过消费速度间接调节 actor 生产速度。

6.4 stale shard 清理

`shard_policy.py` 中的 `discard_stale_shards()` 会根据当前权重版本清理过旧 shard。逻辑要点是：

- shard 文件名必须能解析出 `_v{version}`。
- learner 当前版本记为 `cur_weights_version`。
- 允许保留的最小版本由 `upcoming = cur_version + 1` 与 `max_version_gap` 共同决定。
- 太旧或不可解析版本的 shard 会被直接移动到 `consumed/`。

这使得 learner 不会长期训练明显过时的采样数据。

6.5 replay 兼容性过滤

`discard_incompatible_shards()` 会加载 shard，并调用 `agent.replay_is_compatible()` 检查 replay schema 是否仍可被当前 Agent 解释。如果 replay 结构失效，Learner 会直接丢弃该 shard。

这一步是 replay 协议变更时的重要保险丝。

6.6 shard 选择策略

`select_shards_for_update()` 的策略非常直接：

- 在 `sync` 模式下，必须每个 actor 至少有一个 shard，且每个 actor 只取一个。
- 在 `async` 模式下，只要可用 shard 数达到 `shards_per_update`，就按文件排序取前 `N` 个。

也就是说，当前系统没有更复杂的优先级调度、采样均衡或新鲜度排序策略，核心目标是先保证训练闭环稳定。

7. Learner 侧执行路径

7.1 Learner 初始化

learner_runtime.py 中的 learner_main() 会按如下顺序执行：

1. 初始化分布式环境。
2. 解析 actor-learner 配置。
3. 创建 BufferPaths 并确保目录存在。
4. 检查 async 模式下 shards_per_update 是否超过总 buffer 容量，必要时进行钳制，避免死锁。
5. 构建 Agent。
6. 如果启用 DDP，则对 Agent 执行 wrap_ddp()。
7. 通过 build_algorithm_bundle() 构建算法规格对象、value_net 与算法元信息。
8. 通过 actor_learner_lightning_config_from_algorithm() 生成 learner handoff 配置。
9. rank0 初始化权重版本文件，并把当前 Agent 参数保存为 latest.ckpt。
10. 构建 ActorLearnerLightningModule 与 ActorLearnerUpdateDataModule。
11. 构建 Lightning Trainer 并调用 trainer.fit()。

7.2 版本初始化约定

Learner 启动时，如果 version.txt 不存在或版本小于等于 0，会先：

- 将 version.txt 写为 1
- 将当前 Agent 参数保存到 latest.ckpt

这代表当前训练协议默认认为“初始可用权重版本”为 1，而不是 0。

7.3 算法 bundle 的构建

learner_factory.py 负责构建 learner 训练所需的算法组件。

PPO 路径下：

- 创建策略规格对象 PPO(...)
- 创建 value network
- 如配置允许且 Agent 支持 value features，则用 ValueHead
- 否则退回卷积式 ValueNet

ReinforcePP 路径下：

- 创建 ReinforcePP(...)
- 不构建 value_net

这说明 value baseline 的责任只在 PPO 家族中成立。

7.4 典型配置映射

以 ppo_closed_loop_sparsedrive_v2.yaml 为例，训练配置中的关键映射关系如下：

- train.algo=ppo 决定 learner 走 PPO 路径。
- train.actor_learner.mode=async 决定 shard 选择与 actor 等待策略。
- train.actor_learner.actor_gpu_pool + actors_per_gpu 经规范化后生成 actor GPU 计划。
- train.epochs=2 被映射为 learner 的 inner_epochs=2。
- train.ddp.grad_accum_steps=16 进入 Lightning 的 accumulate_grad_batches。
- train.critic_use_agent_features=true 允许 value net 使用 agent 提供的 feature 向量，而不是原始 obs。
- agent.trainable_prefixes 控制 SparseDriveV2 中真正被微调的参数范围。

8. DataModule 与 batch 构建

8.1 ActorLearnerUpdateDataModule 的角色

actor_learner_datamodule.py 是 Learner update 周期的起点。与普通 datamodule 不同，它不仅提供 dataloader，还负责：

- 等待 shard 到齐。
- 基于当前版本丢弃 stale shard。
- 基于 replay schema 丢弃不兼容 shard。
- 选择本轮 update 使用的 shard。
- 调用 build_training_batch() 组装 batch。
- 在 inner_epochs > 1 时复用同一批 shard 多轮训练。

这里体现了 actor-learner datamodule 与普通监督学习 datamodule 的本质差异：它同时承担了“等待数据”和“准备数据”两种职责。

8.2 一个 update 与多个 inner epoch

代码里有一个关键概念区分：

- update：采样一批 shard，训练完成后保存新权重并推进一次版本号。
- inner epoch：在同一批 shard 上重复训练一次 dataloader 全遍历。

在实现上：

- current_epoch // inner_epochs 对应 update index。
- current_epoch % inner_epochs 对应当前 inner epoch。

这使得 PPO 可以在同一批数据上做多轮优化，同时又不破坏 actor-learner 的 update 语义。

8.3 build_training_batch 的核心逻辑

actor_learner.py 是 learner 数据准备最关键的文件。

PPO 分支

对于每个 shard, 会执行：

- 读取 obs、old_logp、reward、done、terminated、replay
- 用 value_net 对轨迹状态计算 values_i
- 根据 next_obs 或 next_value_feature 计算 bootstrap 的 last_value
- 调用 compute_gae() 计算 adv_i 和 ret_i
- 累积 old_value

Reinforce 分支

- 不构建 value baseline
- 调用 compute_returns() 直接得到 return
- 令 $adv_i = ret_i$

聚合后输出

最终输出的 batch 包含：

- obs_batch
- old_logp
- old_value
- adv
- ret
- replay

同时返回一个 LoadedShardBatch, 其中还记录：

- num_samples
- reward_sum
- reward_count
- done_sum
- done_count

这些统计量后续会被 learner 用于日志和 WandB 上报。

8.4 GAE 计算的实现细节

compute_gae() 的实现有两个容易忽略的细节：

1. terminated 与 done 分开

done 表示 episode 是否结束, 可能由 terminated 或 truncated 触发。

bootstrap 时使用的是 not_terminated, 而不是简单 not_done, 这样 timeout/截断场景下仍可允许 bootstrap。

2. 递推项使用 not_done

$last_gae = \delta + \gamma * \lambda * not_done * last_gae$

这保证 episode 断开后不会把后续轨迹优势泄漏到前一段。

GAE 的核心递推代码如下：

```
for t in reversed(range(horizon)):    not_done = 1.0 - dones[t]    not_terminated = 1.0 - terminated[t]
```

这段代码最值得注意的是 not_done 和 not_terminated 的分离。它意味着框架把“轨迹是否中断”和“是否允许 bootstrap”视为两个不同问题来处理，这在包含 timeout、截断、失败终止等自动驾驶场景里是很有必要的。

8.5 advantage 标准化

normalize_advantages() 支持两种模式：

- 单卡：本地均值方差标准化。
- DDP：先通过 all_reduce 聚合全局和、平方和、样本数，再做全局标准化。

这确保多卡训练时各 rank 看到的优势归一化尺度一致。

9. Lightning 训练步

9.1 configure_optimizers 的权责

trajectory_module.py 里的 configure_optimizers() 是当前优化器构建的权威入口。

它会：

- 从 policy_module 或 agent 中提取可训练参数。
- 用 policy_lr 构建策略参数组。
- 如果是 PPO，再为 value_net 单独构建 value 参数组。
- 最终统一返回 torch.optim.Adam。

这体现了仓库当前推荐方向：优化器所有权在 Lightning，而不是继续停留在 runner 或算法对象里。

9.2 training_step 的实际流程

对于每个 minibatch，training_step() 执行以下步骤：

1. 从 batch 中取出 replay、adv、ret、old_logp。
2. 调用 agent_logp_from_replay_batch() 重算当前参数下的 new_logp。
3. 如果算法是 PPO：
准备 value input 前向计算 value_pred 调用 compute_ppo_objective() 调用 compute_ppo_metrics()
4. 如果算法是 Reinforce：
调用 compute_reinforce_objective() 调用 compute_reinforce_metrics()
5. 如果配置了 distillation teacher，则额外计算 forward KL / reverse KL 并加到总 loss 上。
6. 记录最新 metrics、累积 update 内 metrics、写 Lightning log。

Learner 真正执行优化的核心代码摘录如下：

```
new_logp = agent_logp_from_replay_batch(    self.agent,    replay,    device=device,    eta=float(self.learn
```

这段代码说明 learner 的中心动作其实非常简单直接：基于 replay 重算 new_logp，然后把 old_logp / adv / ret / value_pred 送入对应目标函数。也就是说，当前框架真正的训练核心不是“重新跑一次环境”，而是“利用 replay 在当前参数下重建策略概率”。

9.3 replay 是训练步的中心

这套架构里最核心的训练接口不是“直接把动作 logits 存在 shard 里”，而是：

- Actor 存 replay
- Learner 根据 replay 重算新策略下的 log-prob

因此，replay schema 的稳定性直接决定 learner 是否还能正确训练旧 shard。

9.3.1 当前主路径下 replay 具体包含什么

在当前主路径中，配置文件 ppo_closed_loop_sparsedrive_v2.yaml 使用的是 agent.type = sparsedrive_v2，因此 training_step() 里看到的 replay，其真实结构以 policy_sparsedrive_v2.py 为准。

这里要先区分两个层次：

- shard 是完整训练样本，里面有 reward、done、old_logp、obs、replay 等字段。
- replay 不是整条轨迹，而是“Learner 重算当前策略 log-prob 和 value feature 所需的模型侧最小上下文”。

当前 SparseDriveV2 的 replay 字段主要包括：

1. camera_feature

来源：Actor 采样时由 _build_features() 从 observation 提取。

内容：至少包含 imgs，还会携带相机几何相关 meta。

用途：Learner 在 logp_from_replay_batch() 中重新拼成 batch，再次送入策略网络前向。

是否核心：是。

2. status_feature

来源：Actor 采样时由 _build_status_feature() 生成。

内容：一个 1x8 状态向量；优先取 observation 中的 ego_status，否则退化为 driving_command(4) + ego_velocity(2) + ego_acceleration(2)。

用途：和 camera_feature 一起作为策略前向输入；critic 若走 agent feature 路径，也会依赖它。

是否核心：是。

3. mode_idx

来源：Actor 采样时从 candidate modes 中实际选中的索引。

内容：一个整数。

用途：Learner 在重新前向得到所有 mode 的 logits 后，用它取出当时被执行那条 mode 的 log-prob。

是否核心：是。

4. traj_xyyaw

来源：Actor 当时选中的候选轨迹。

内容：轨迹点序列，通常是 (T, 3)，表示 x, y, yaw。

用途：主要用于 actor 动作执行和调试回看，不是 PPO loss 的直接输入。

是否核心：否。

5. candidate_scores

来源：Actor 前向时输出的候选轨迹分数。

内容：所有候选 mode 的打分。

用途：主要用于调试、分析选模行为；当前 learner 重算 log-prob 时不会直接依赖这份缓存分数。

是否核心：否。

6. execute_mode

来源：Actor 当前执行模式配置。

内容：例如 first_step。

用途：主要用于采样行为解释和调试。

是否核心：否。

7. feature_missing_fields

来源：构建特征时发现 observation 缺字段时记录。

内容：缺失字段名列表。

用途：帮助排查观测侧问题，不直接进入损失计算。

是否核心：否。

可以把当前 SparseDriveV2 的 replay 理解为下面这个简化结构：

```
replay = { "camera_feature": {...}, "status_feature": tensor(...), "mode_idx": int(...), "traj_x": ... }
```

其中真正被 learner 核心训练路径直接消费的是：

- camera_feature
- status_feature
- mode_idx

这是因为在 policy_sparsedrive_v2.py 中：

- logp_from_replay_batch() 依赖 camera_feature + status_feature + mode_idx
- value_features_from_replay_batch() 依赖 camera_feature + status_feature

这也解释了为什么文档前面会强调 replay 是“模型侧重放上下文”，而不是环境侧监督标签。

9.3.2 replay 不包含什么

为了避免误解，这里也明确列出当前 replay 不负责承载的内容：

- reward 不在 replay 里，而是在 shard 顶层。
- done / terminated / truncated 不在 replay 里，而是在 shard 顶层。
- old_logp 不在 replay 里，而是在 shard 顶层单独存储。
- adv / ret / old_value 不在 replay 里，而是 learner 读取 shard 后再构建出来的。

所以更准确地说：

- shard 负责保存训练样本整体
- replay 负责保存“如何重新跑策略前向”

9.3.3 不同 Agent 的 replay 会不一样

虽然 training_step() 统一把它们都叫 replay，但不同 Agent 的 replay schema 实际并不相同。

例如：

- policy_diffusiondrivev2.py 的 replay 会包含 camera_feature、status_feature、diffusion_chain、mode_idx、traj_xyyaw
- policy_sparsedrive.py 的 replay 会保存 img、相机投影与姿态信息、ego_status、gt_ego_fut_cmd、cmd_idx、mode_idx、traj_xyyaw

因此，replay 的统一性体现在“都能让 Agent 重新算 log-prob”，而不是“字段名完全一致”。这也是为什么 learner 在消费 shard 前，需要通过 agent.replay_is_compatible() 过滤不兼容 replay。

9.4 PPO 目标函数细节

trajectory_policy_core.py 中 compute_ppo_objective() 的核心计算包括：

- $\text{ratio} = \exp(\text{new_logp} - \text{old_logp})$
- $\text{surr1} = \text{ratio} * \text{adv}$
- $\text{surr2} = \text{clamp}(\text{ratio}, 1 - \text{clip_eps}, 1 + \text{clip_eps}) * \text{adv}$
- $\text{loss_pi} = -\text{mean}(\min(\text{surr1}, \text{surr2}))$
- 若配置了 value clip，则 value loss 用 clipped/unclipped 中更大的那个
- $\text{loss} = \text{loss_pi} + \text{vf_coef} * \text{loss_v} + \text{kl_coef} * \text{approx_kl}$

如果开启 dual_clip，则在 $\text{adv} < 0$ 时进一步使用 dual clip 保护。

9.5 ReinforcePP 目标函数细节

compute_reinforce_objective() 有两种行为：

- 如果 batch 中有 old_logp，则走带 ratio clip 的变体。
- 如果没有 old_logp，则退化为标准 $-(\text{adv} * \text{new_logp}).\text{mean}()$ 。

这使 Reinforce 路径既能兼容更传统的 REINFORCE，也能支持带旧策略参考的近端更新变体。

10. Update 生命周期与权重发布

10.1 on_train_epoch_start

actor_learner_module.py 在 on_train_epoch_start() 中做了两件关键事：

- 如果当前 epoch 没有数据，则设置 `trainer.should_stop=True`
- 如果这是一个 update 的开始，并且当前 rank 是 0，则写入 `TRAINING_LOCK`

因此 `TRAINING_LOCK` 的语义不是“整个训练进程启动了”，而是“learner 当前正处在 update 训练窗口中”。

10.2 on_train_epoch_end

当一个 update 的最后一个 inner epoch 结束时，rank0 会执行：

1. 删除 TRAINING_LOCK
2. 将本轮使用的 shard 全部移动到 consumed/
3. 调用 prune_consumed() 保留本轮相关 shard, 清理更早的 consumed 文件
4. 读取当前版本号并加一
5. 将 Agent 权重保存到 latest.ckpt
6. 把新版本写入 version.txt

这一步同时完成了“消费确认”和“新模型发布”。

版本推进的核心实现如下：

```
for fp in selected:    move_to_consumed(self.paths, fp)prune_consumed(self.paths, keep_basenames={os.path.ba
```

这里可以看到当前协议的一个关键顺序约束：先保存新 checkpoint, 再写新版本号。Actor 的热更新逻辑默认信任这个顺序, 因此如果未来调整发布机制, 必须保证“version 不会先于可加载 checkpoint 可见”这个约束仍然成立。

10.3 指标记录

更新结束后, 模块会汇总：

- shard 数量
- sample 数量
- reward 均值
- done rate
- return 均值与方差
- advantage 方差
- collect / train / save 等阶段耗时
- PPO / Reinforce 的损失与 KL 指标

这些信息会通过 stage_fn() 打印, 并在启用 WandB 时写入 train_update/* 命名空间。

11. Agent 适配层的工程意义

11.1 build_agent 的分发逻辑

agent_factory.py 根据 agent.type 决定构建哪种 Agent：

- dummy
- sparsedrive
- sparsedrive_v2
- 默认 diffusiondrive_v2

其中 policy_execute_mode 会被规范化到 first_step, 说明当前主训练路径默认采用“规划轨迹首步执行”的动作落地方式。

11.2 DummyPolicy 的参考价值

policy_dummy.py 尽管不参与真实训练，但它非常适合作为理解 Agent 协议的最小样例。它展示了：

- 如何把 observation 转成 replay。
- 如何实现 act 与 act_batch。
- 如何基于 replay 重新计算 log-prob。
- 如何最小化实现 checkpoint 的 save/load。

对于新策略接入，这个文件是最容易读懂的模板。

11.3 SparseDriveV2 的接入方式

虽然本文没有逐行展开 policy_sparsedrive_v2.py，但从 agent_factory.py 和配置文件可以确认，当前主路径对 SparseDriveV2 的接入具有几个特点：

- 支持通过 trainable_prefixes 精确指定可训练参数子树。
- 训练时并不是粗粒度地“整个模型一起训”，而是偏微调式接入。
- replay 既承担 learner 侧重算 log-prob 的职责，也承担 value feature 提取的潜在输入职责。

12. 环境与奖励路径

12.1 场景发现与分片

env_factory.py 负责：

- 从数据目录自动发现合法 scene_id
- 根据 scene_shard_by_actor 和 scene_shard_strategy 把场景集分配给不同 actor

当前支持两种策略：

- round_robin
- contiguous

这意味着多 actor 并行不仅是并行采样，更是显式的数据源切分。

12.2 环境包装层的职责

根据 env_wrapper/README.md，环境包装层会把底层仿真统一为 RL 的 obs, reward, terminated, truncated, info 协议，同时把奖励和终止逻辑纳入同一层。

12.3 奖励配置

以当前 SparseDriveV2 PPO 配置为例，reward 配置可直接控制：

- 位置误差权重
- 航向误差权重
- 静态/动态碰撞权重

- 纵向 jerk / yaw jerk 惩罚
- terminal penalty 触发条件

也就是说，reward 行为主要由配置驱动，而不是写死在训练循环中。

13. 重要训练契约与隐含约束

13.1 协议级产物

以下产物已经具有协议意义，不建议随意修改：

- buffer/shards/*.pt
- buffer/consumed/*.pt
- weights/latest.ckpt
- weights/version.txt
- STOP
- TRAINING_LOCK

这些文件名和用途同时被 actor、learner、orchestrator 三方依赖。

13.2 shard schema 契约

Learner 当前默认依赖 shard 至少包含：

- reward
- done
- replay

PPO 额外强依赖：

- obs
- old_logp
- next_obs 或等价 bootstrap 信息

如果修改 shard schema，必须同步检查：

- rollout 侧写入逻辑
- batch 侧读取逻辑
- replay 兼容性校验
- algorithm/lightning 训练步输入假设

13.3 版本推进语义

当前 learner 的权重发布语义是：

- 一个 update 完成后，版本号加 1

- latest.ckpt 始终对应当前 version.txt

Actor 的热更新逻辑默认信任这个契约，因此不能出现“先写版本号再写权重”的破坏性变更。

13.4 当前实现上的一个显式限制

actor_runtime.py 中虽然保留了 vector env 路径，但如果 num_envs_per_actor !=

1，代码会先打印日志并强制设回 1。这说明：

- 当前主路径的并行主要依赖“多 actor 多进程”，而不是“单 actor 多环境”。
- vector env 支持仍是保留能力，但不是当前主要使用形态。

14. 设计优点、局限与演进建议

14.1 当前设计优点

- 角色职责清晰，orchestrator、actor、learner 边界明确。
- 文件协议简单可观测，训练问题容易落到具体目录和文件排查。
- Agent 抽象清晰，新策略接入时无需改动主链路大部分模块。
- Lightning 已经承担训练步、优化器和 update 生命周期，训练所有权比过去更清晰。
- 数据从环境到算法的流向相对干净，路径可读性强。

14.2 当前局限

- shard 选择策略仍较朴素，没有显式新鲜度排序、重要性采样或 actor 公平性优化。
- buffer 介质是文件系统，简单稳定，但吞吐和元数据管理能力有限。
- vector env 路径当前没有作为主形态跑通。
- Actor 和 Learner 的异常恢复机制仍偏轻量，更多依赖 STOP/重启，而不是强状态机恢复。
- replay schema 的稳定性高度依赖各模型适配器实现，若接入更多模型，兼容性治理成本会继续上升。

14.3 建议的后续演进方向

- 为 shard schema 和 replay schema 增加显式版本号。
- 为 select_shards_for_update() 增加更细的调度策略，例如优先最新版本、控制 actor 覆盖率等。
- 增加 actor-learner smoke test，覆盖版本发布、stale shard 丢弃和 replay 不兼容处理。
- 继续收敛 runner 与 lightning 的职责，避免再次出现训练所有权回流到 runtime 层。
- 如果未来吞吐要求提高，可评估把 buffer

从文件系统升级为更结构化的存储层，但前提是先明确恢复和可观测性方案。

15. 总结

ReconDreamer-RL/framework 当前已经完成一个可运行、可扩展、边界相对清晰的 actor-learner RL 训练主框架。它的核心价值不是单个算法实现，而是建立了一套稳定的工程训练闭环：

- Actor 负责采样。
- Learner 负责更新。
- Buffer 负责解耦。
- Version 负责发布。
- Agent 负责屏蔽模型差异。
- Lightning 负责承接训练生命周期。

从工程角度看，这套框架“职责分层清楚、协议简单、调试路径明确”。后续无论是补充新模型、扩展新算法，还是增强训练稳定性，都应该尽量沿着现有分层推进，而不是重新把逻辑耦合回入口脚本或单个超大模块中。